

§ 12.2 *选取与中位数

12.2.1 概述

■ k-选取

考查如下问题：

在任意一组可比较大小的元素中，如何找出由小到大次序为 k 者？

如图12.7(a)所示，也就是要从与这组元素对应的有序序列 S 中，找出秩为 k 的元素 $S[k]$ ，故称作选取（selection）问题。若将目标元素的秩记作 k ，则亦称作 k -选取（ k -selection）问题。以无序向量 $A = \{ 3, 13, 2, 5, 8 \}$ 为例，对应的有序向量为 $S = \{ 2, 3, 5, 8, 13 \}$ ，其中的元素依次与 $k = \{ 0, 1, 2, 3, 4 \}$ 相对应。



图12.7 选取与中位数

作为 k -选取问题的特例， 0 -选取即通常的最小值问题，而 $(n - 1)$ -选取问题即通常的最大值问题。这两个问题都有平凡的最优解，例如`List::selectMax()`（82页代码3.21）。

在允许元素重复的场合，秩为 k 的元素可能同时存在多个副本。此时不妨约定，其中任何一个都可作为解答输出。

■ 中位数

如图12.7(b)所示，在长度为 n 的有序序列 S 中，位序居中的元素 $S[\lfloor n/2 \rfloor]$ 称作中值或中位数（median）。例如，有序序列 $S = \{ 2, 3, \boxed{5}, 8, 13 \}$ 的中位数，为 $S[\lfloor 5/2 \rfloor] = S[2] = 5$ ；而有序序列 $S = \{ 2, 3, 5, \boxed{8}, 13, 21 \}$ 的中位数，则为 $S[\lfloor 6/2 \rfloor] = S[3] = 8$ 。

即便对于尚未排序的序列，也可定义中位数——也就是在对原数据集排序之后，对应的有序序列的中位数。例如，无序序列 $A = \{ 3, 13, 2, \boxed{5}, 8 \}$ 的中位数为元素 $A[3] = 5$ 。

由于中位数可将原数据集（原问题）划分为大小明确、规模相仿且彼此独立的两个子集（子问题），故能否高效地确定中位数，将直接关系到采用分治策略的算法能否高效地实现。

■ 蛮力算法

由中位数的定义，可直接得到查找中位数的如下直觉算法：对所有元素做排序，将其转换为有序序列 S ；于是， $S[\lfloor n/2 \rfloor]$ 便是所要找的中位数。然而根据2.7.5节的结论，该算法在最坏情况下需要 $\Omega(n \log n)$ 时间。于是，基于该算法的任何分治算法，时间复杂度都会不低于：

$$T(n) = n \log n + 2 \cdot T(n/2) = \Theta(n \log^2 n)$$

这一效率难以令人接受。

综上所述，中位数查找问题的挑战恰恰就在于：

如何在避免全排序的前提下，在 $\Theta(n \log n)$ 时间内找出中位数？

不难看出，所谓中位数查找问题，也可以理解为是选取问题在 $k = \lfloor n/2 \rfloor$ 时的特例。稍后我们将看到，中位数查找问题既是选取问题的特例，同时也是选取问题中的难度最大者。

以下先结合若干特定情况讨论中位数的定位算法，然后再回到一般性的选取问题。

12.2.2 众数

■ 问题

为达到热身的目的，不妨先来讨论中位数问题的一个简化版本。在任一无序向量A中，若有一半以上元素的数值同为m，则将m称作A的众数（majority）。例如，向量{ 5, 3, 9, 3, 3, 2, 3, 3 }的众数为3；而虽然3在向量{ 5, 3, 9, 3, 1, 2, 3, 3 }中最多，确非众数。

那么，任给无序向量，如何快速判断其中是否存在众数，并在存在时将其找出？尽管只是以整数向量为例，以下算法不难推广至元素类型支持判等和比较操作的任意向量。

■ 必要性与充分性

不难理解但容易忽略的一个事实是：若众数存在，则必然同时也是中位数。否则，在对应的有序向量中，总数超过半数的众数必然被中位数分隔为非空的两组——与向量的有序性相悖。

```
1 template <typename T> bool majority(Vector<T> A, T& maj) { //众数查找算法：T可比较可判等
2     maj = majEleCandidate(A); //必要性：选出候选者maj
3     return majEleCheck(A, maj); //充分性：验证maj是否的确当选
4 }
```

代码12.4 众数查找算法主体框架

因此可如代码12.4所示，通过调用majEleCandidate()，从向量A中找到中位数maj（如果的确可以高效地查找到的话），并将其作为众数的唯一候选者。

然后再如代码12.5所示，调用majEleCheck()在线性时间内扫描一遍向量，通过统计该中位数出现的次数，即可验证其作为众数的充分性，从而最终判断向量A的众数是否的确存在。

```
1 template <typename T> bool majEleCheck(Vector<T> A, T maj) { //验证候选者是否确为众数
2     int occurrence = 0; //maj在A[]中出现的次数
3     for (int i = 0; i < A.size(); i++) //逐一遍历A[]的各个元素
4         if (A[i] == maj) occurrence++; //每遇到一次maj，均更新计数器
5     return 2 * occurrence > A.size(); //根据最终的计数值，即可判断是否的确当选
6 }
```

代码12.5 候选众数核对算法

那么，在尚未得到高效的中位数查找算法之前，又该如何解决众数问题呢？

■ 减而治之

关于众数的另一重要事实，如图12.8所示：

设P为向量A中长度为2m的前缀。若元素x在P中恰好出现m次，则A有众数仅当后缀A-P拥有众数，且A-P的众数就是A的众数。

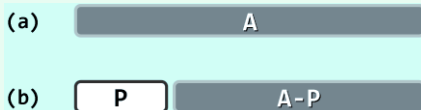


图12.8 通过减治策略计算众数

既然最终总会针对充分性另作一次核对，故不必担心A不含众数的情况，而只需验证A的确拥有众数的两种情况。若A的众数就是x，则在剪除前缀P之后，x与非众数均减少相同的数目，二者数目的差距在后缀A-P中保持不变。反过来，若A的众数为 $y \neq x$ ，则在剪除前缀P之后，y减少的数目也不致多于非众数减少的数目，二者数目的差距在后缀A-P中也不会缩小。